

Techniques de requête optimisées



Samuel Rohaut

Stratégies de jointure optimisées

Stratégies de jointure optimisées

Nested Loop Join

Mécanisme : Pour chaque ligne de la table externe, parcours chaque ligne de la table interne

Idéal pour : Petites tables, jointures avec index, prédicats très sélectifs

Exemple de plan : Nested Loop → Index Scan

Stratégies de jointure optimisées

Hash Join

Mécanisme : Création d'une table de hachage en mémoire avant jointure

Idéal pour : Grandes tables sans index appropriés, conditions d'égalité

Exemple de plan : Hash Join $\rightarrow$ Hash $\rightarrow$ Seq Scan

Stratégies de jointure optimisées

Merge Join

Mécanisme : Fusion de deux ensembles préalablement triés

Idéal pour : Tables avec index sur les colonnes de jointure

Exemple de plan : Merge Join $\rightarrow$ Sort $\rightarrow$ Seq Scan

Stratégies de jointure optimisées

Ordre des jointures

Impact crucial sur les performances

Règle générale : Commencer par les tables les plus filtrées

Contrôle avec jointures explicites (RIGHT, LEFT, INNER)

Stratégies de jointure optimisées

Jointures latérales (LATERAL)

Permettent aux sous-requêtes d'accéder aux colonnes des tables précédentes

Applications : Top-N par groupe, requêtes complexes dépendantes

Remplacent souvent des boucles applicatives

Exemple

```
-- Approche optimisée: commencer par la condition la plus restrictive
SELECT c.customer_name, o.order_date, p.product_name
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
WHERE o.order_date between '2023-01-01' and '2023-01-02'
      AND p.category = 'Electronics'
      AND c.region = 'Europe';
```

Cas d'utilisation: Système de commerce électronique devant générer des rapports de ventes régionaux où seulement certaines catégories de produits et périodes sont pertinentes. En commençant par les tables avec les filtres les plus restrictifs, on réduit rapidement le volume de données à traiter.

**Sous-requêtes
efficaces**

Sous-requêtes efficaces

Types de sous-requêtes

Non-corrélées : Exécutées une seule fois, indépendantes

Corrélées : Exécutées pour chaque ligne, référencent la requête externe

Sous-requêtes efficaces

Choix des opérateurs

EXISTS vs. IN :

EXISTS souvent plus performant pour les grands ensembles

IN meilleur pour petits ensembles de valeurs distinctes

ANY/ALL :

Optimisés pour comparaisons avec ensembles

Parfois remplaçables par MIN/MAX

Sous-requêtes efficaces

Transformation en jointures

Les sous-requêtes peuvent être réécrites en jointures

Avantages : Plus d'options d'optimisation pour le planificateur

Réécriture manuelle parfois nécessaire

Exemple

```
-- Version moins efficace avec IN
SELECT department_name, department_id
FROM departments
WHERE department_id IN (
    SELECT department_id
    FROM employees
    WHERE salary > 50000
);

-- Version optimisée avec EXISTS
SELECT department_name, department_id
FROM departments d
WHERE EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.department_id = d.department_id
    AND e.salary > 50000
);
```

Cas d'utilisation: Système de commerce électronique devant générer des rapports de ventes régionaux où seulement certaines catégories de produits et périodes sont pertinentes. En commençant par les tables avec les filtres les plus restrictifs, on réduit rapidement le volume de données à traiter.

Common Table Expressions (WITH)

Common Table Expressions (WITH)

CTEs simples

Structuration des requêtes complexes

Référence multiple à un même résultat intermédiaire

Matérialisation vs. inlining

Avant PG 12 : Toujours matérialisées

PG 12+ : Option explicite (MATERIALIZED / NOT MATERIALIZED)

Impact : Performance vs. utilisation multiple

Common Table Expressions (WITH)

CTEs récursives

Traitement hiérarchique (organigrammes, catégories imbriquées)

Parcours de graphes (réseaux, chemins)

Génération de données (séquences, dates)

Exemple

```
WITH RECURSIVE org_hierarchy AS (  
  -- Cas de base (dirigeants sans manager)  
  SELECT employee_id, name,  
         position, 1 AS level  
  FROM employees  
  WHERE manager_id IS NULL  
  
  UNION ALL  
  
  -- Récursion (tous les subordonnés directs)  
  SELECT e.employee_id, e.name,  
         e.position, oh.level + 1  
  FROM employees e  
  JOIN org_hierarchy oh  
    ON e.manager_id = oh.employee_id  
)  
SELECT level, name, position  
FROM org_hierarchy  
ORDER BY level, name;
```

Cas d'utilisation: Visualisation d'un organigramme d'entreprise avec niveaux hiérarchiques. Cette approche récursive permet de parcourir l'arborescence complète sans connaître sa profondeur à l'avance, évitant des jointures multiples ou du code applicatif complexe.

Window Functions

Window Functions

Syntaxe fondamentale

OVER () : Définition de la fenêtre

PARTITION BY : Segmentation des données

ORDER BY : Ordonnancement dans la fenêtre

Window Functions

Fonctions principales

Agrégation : SUM, AVG, COUNT sur des fenêtres

Rang : ROW_NUMBER, RANK, DENSE_RANK

Position : LEAD, LAG, FIRST_VALUE, LAST_VALUE

Distribution : NTILE, PERCENT_RANK

Window Functions

Applications pratiques

Calculs cumulatifs sans auto-jointures

Comparaisons avec valeurs précédentes/suivantes

Classements et percentiles

Exemple

```
SELECT
    product_name,
    category,
    monthly_sales,
    SUM(monthly_sales) OVER (PARTITION BY category) AS category_total,
    ROUND(monthly_sales * 100.0 / SUM(monthly_sales)
        OVER (PARTITION BY category), 2) AS percent_of_category,
    RANK() OVER (PARTITION BY category
        ORDER BY monthly_sales DESC) AS rank_in_category,
    SUM(monthly_sales) OVER (PARTITION BY category
        ORDER BY monthly_sales DESC) AS running_total
FROM product_sales
WHERE sales_month = '2023-03';
```

Cas d'utilisation: Tableau de bord analytique de ventes permettant de voir simultanément les ventes totales par catégorie, la contribution de chaque produit (en pourcentage), son classement et la somme cumulée. Sans window functions, cela nécessiterait plusieurs requêtes ou auto-jointures complexes.

Agrégation et groupement avancés

Agrégation et groupement avancés

Techniques multidimensionnelles

ROLLUP : Totaux hiérarchiques (ex: Pays > Région > Ville)

CUBE : Totaux selon toutes les combinaisons possibles

GROUPING SETS : Sélection précise des niveaux d'agrégation

Agrégation et groupement avancés

Optimisation des agrégations

Filtrage pré-agrégation (WHERE) vs post-agrégation (HAVING)

Utilisation des index pour accélérer GROUP BY

Agrégations partielles avec CTEs

Agrégation et groupement avancés

Agrégation conditionnelle

`FILTER:SUM(...) FILTER (WHERE condition)`

`CASE:SUM(CASE WHEN ... THEN ... END)`

Exemple

```
SELECT
    COALESCE(region, 'Total') AS region,
    COALESCE(country, 'Total') AS country,
    COALESCE(TO_CHAR(order_date, 'YYYY-MM'), 'Total')
        AS month,
    SUM(sales_amount) AS total_sales,
    GROUPING(region, country,
        TO_CHAR(order_date, 'YYYY-MM')) AS grouping_level
FROM sales
WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31'
GROUP BY GROUPING SETS (
    (region, country, TO_CHAR(order_date, 'YYYY-MM')),
    (region, country),
    (region),
    ()
)
ORDER BY
    GROUPING(region), region,
    GROUPING(country), country,
    GROUPING(TO_CHAR(order_date, 'YYYY-MM')), month;
```

Cas d'utilisation: Rapport financier consolidé montrant à la fois les ventes détaillées par région/pays/mois et les différents niveaux d'agrégation (totaux par pays, région et global). En une seule requête, on obtient ce qui aurait nécessité plusieurs UNION ALL.

Pagination et tri optimisés

Pagination et tri optimisés

Problèmes de LIMIT/OFFSET

Performance dégradée pour pages éloignées

Inefficacité liée au traitement+rejet de lignes

Pagination et tri optimisés

Pagination par valeur-clé (keyset pagination)

Utilisation des valeurs de la dernière ligne

Stabilité malgré les modifications

Performances constantes quelle que soit la page

Pagination et tri optimisés

Optimisation du tri

- Alignement avec les index

- Minimisation de la taille des données à trier

- Utilisation de tri partiel quand possible

Exemple

```
-- Pagination traditionnelle
-- (problématique pour pages éloignées)
SELECT id, title, created_at
FROM articles
ORDER BY created_at DESC, id
LIMIT 20 OFFSET 1000;

-- Pagination par valeur-clé (keyset) optimisée
SELECT id, title, created_at
FROM articles
WHERE (created_at, id) < ('2023-04-15', 12345)
    -- Dernières valeurs de la page précédente
ORDER BY created_at DESC, id
LIMIT 20;
```

Cas d'utilisation: Fil d'actualités ou blog avec volume important d'articles. La pagination par valeur-clé maintient des performances constantes quelle que soit la "profondeur" de la page, contrairement à OFFSET qui doit parcourir et rejeter toutes les lignes précédentes.

Techniques de matérialisation

Techniques de matérialisation

Vues matérialisées

Stockage physique des résultats d'une requête

Rafraîchissement configurable (REFRESH MATERIALIZED VIEW)

Indexation des résultats matérialisés

Techniques de matérialisation

Tables temporaires et de travail

Tables temporaires de session vs transaction

Tables dédiées aux calculs intermédiaires

Préparation des données pour traitement ultérieur

Techniques de matérialisation

Quand matérialiser

Requêtes coûteuses exécutées fréquemment

Agrégations complexes sur données relativement stables

Préparation d'analyses en plusieurs étapes

Exemple

```
-- Création d'une vue matérialisée
-- pour des calculs fréquents et coûteux
CREATE MATERIALIZED VIEW monthly_sales_stats AS
SELECT
    DATE_TRUNC('month', order_date) AS month,
    product_category,
    region,
    COUNT(DISTINCT customer_id) AS unique_customers,
    SUM(quantity) AS total_units,
    SUM(amount) AS total_sales,
    AVG(amount) AS avg_order_value
FROM orders
JOIN order_items USING (order_id)
JOIN products USING (product_id)
GROUP BY 1, 2, 3
WITH DATA;

-- Création d'index sur la vue matérialisée
CREATE INDEX idx_monthly_sales_category
    ON monthly_sales_stats(product_category);
CREATE INDEX idx_monthly_sales_region
    ON monthly_sales_stats(region);

-- Rafraîchissement périodique
REFRESH MATERIALIZED VIEW monthly_sales_stats;
```

Cas d'utilisation: Tableau de bord décisionnel consulté fréquemment par de nombreux utilisateurs, avec des données mises à jour quotidiennement. La vue matérialisée pré-calcule les agrégations complexes, permettant des consultations instantanées.

Partitionnement et requêtage

Partitionnement et requêtage

Élagage des partitions (partition pruning)

Élimination automatique des partitions non pertinentes

Conditions d'élagage explicites

Vérification via EXPLAIN (prune=false)

Partitionnement et requêtage

Stratégies efficaces

Alignement entre partitionnement et requêtes fréquentes

Granularité adaptée au volume et aux patterns d'accès

Complémentarité avec l'indexation

Partitionnement et requêtage

Considérations pour requêtes

Inclusion explicite de la colonne de partitionnement

Utilisation de prédicats compatibles avec l'élagage

Attention aux opérations intégrant toutes les partitions

Exemple

```
-- Définition d'une table partitionnée par intervalle (date)
CREATE TABLE logs (
    log_id SERIAL,
    log_time TIMESTAMPTZ NOT NULL,
    log_level TEXT,
    message TEXT,
    source TEXT
) PARTITION BY RANGE (log_time);

-- Création des partitions
CREATE TABLE logs_2023_q1 PARTITION OF logs
    FOR VALUES FROM ('2023-01-01') TO ('2023-04-01');
CREATE TABLE logs_2023_q2 PARTITION OF logs
    FOR VALUES FROM ('2023-04-01') TO ('2023-07-01');
-- ... autres partitions ...

-- Requête optimisée exploitant l'élagage de partition
SELECT log_time, message
FROM logs
WHERE log_time BETWEEN '2023-03-15' AND '2023-03-20'
AND log_level = 'ERROR';
```

Cas d'utilisation: Système de journalisation à haut volume (millions d'entrées par jour) où les requêtes portent généralement sur des périodes spécifiques. Le partitionnement par date permet au planificateur de cibler uniquement les partitions pertinentes, réduisant drastiquement le volume de données à scanner.

Requêtes parallèles

Requêtes parallèles

Mécanisme et configuration

Paramètres clés : `max_parallel_workers`, `max_parallel_workers_per_gather`

Seuils de déclenchement : `min_parallel_table_scan_size`

Vérification : Nombre de workers dans EXPLAIN

Requêtes parallèles

Opérations parallélisables

Scan séquentiel et d'index

Agrégations (GROUP BY)

Certaines jointures (Hash Join principalement)

Tri (dans certaines conditions)

Requêtes parallèles

Optimisation du parallélisme

Dimensionnement adapté aux ressources

Éviter les barrières au parallélisme

Requêtes conçues pour exploiter le parallélisme

Exemple

```
-- Configuration pour autoriser
-- plus de workers parallèles
SET max_parallel_workers_per_gather = 4;

-- Requête susceptible de bénéficier du parallélisme
EXPLAIN (ANALYZE, BUFFERS)
SELECT
    product_category,
    COUNT(*),
    AVG(amount),
    PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY amount)
FROM large_sales_table
WHERE sale_date > CURRENT_DATE - INTERVAL '1 year'
GROUP BY product_category;
```

Cas d'utilisation: Analyse de données historiques sur un serveur multi-cœurs puissant. La requête parallèle répartit le travail entre plusieurs workers, réduisant significativement le temps d'exécution pour des analyses ad hoc sur des tables volumineuses.

Optimisation par l'analyse de plan

Optimisation par l'analyse de plan

EXPLAIN approfondi

Options avancées : ANALYZE, BUFFERS, WAL

Interprétation fine des coûts et temps

Identification des écarts entre estimation et réalité

Optimisation par l'analyse de plan

Indicateurs de problèmes

Scans séquentiels non nécessaires

Estimations de cardinalité très inexactes

Jointures avec stratégies sous-optimales

Opérations de tri ou agrégation coûteuses

Optimisation par l'analyse de plan

Approche méthodique

Identification des requêtes problématiques (pg_stat_statements)

Analyse systématique du plan

Optimisation progressive et mesurable

Validation par benchmarking

Exemple

```
-- Analyser une requête sous-performante
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.customer_name, COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
WHERE c.region = 'North America'
      AND o.status = 'Completed'
GROUP BY c.customer_id, c.customer_name;

-- Correction possible (préférer filtrage avant jointure)
EXPLAIN (ANALYZE, BUFFERS)
SELECT c.customer_name, COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN (
    SELECT order_id, customer_id
    FROM orders
    WHERE status = 'Completed'
) o ON c.customer_id = o.customer_id
WHERE c.region = 'North America'
GROUP BY c.customer_id, c.customer_name;
```

Cas d'utilisation:

Optimisation d'un rapport de fidélité client qui s'exécute trop lentement. L'analyse du plan révèle que la condition sur la table orders est appliquée trop tard, après la jointure. La correction déplace le filtre dans une sous-requête, réduisant drastiquement le volume de données à joindre.

En résumé

L'optimisation des requêtes est un processus itératif

Importance de comprendre les mécanismes sous-jacents

Équilibre entre réactivité des requêtes et charge système

L'expérience et la mesure sont essentielles pour progresser